

zkVC: Fast Zero-Knowledge Proof for Private and Verifiable Computing

Yancheng Zhang[†], Mengxin Zheng[†], Xun Chen[‡]
Jingtong Hu[§], Weidong Shi^{††}, Lei Ju^{‡‡}, Yan Solihin[†], Qian Lou^{†*}

[†]University of Central Florida, [‡]Samsung Research America,
[§]University of Pittsburgh, ^{††}University of Houston, ^{‡‡}Shandong University

Abstract—In the context of cloud computing, services are held on cloud servers, where the clients send their data to the server and obtain the results returned by server. However, the computation, data and results are prone to tampering due to the vulnerabilities on the server side. Thus, verifying the integrity of computation is important in the client-server setting. The cryptographic method known as Zero-Knowledge Proof (ZKP) is renowned for facilitating private and verifiable computing. ZKP allows the client to validate that the results from the server are computed correctly without violating the privacy of the server’s intellectual property. Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zkSNARKs), in particular, has been widely applied in various applications like blockchain and verifiable machine learning. Despite their popularity, existing zkSNARKs approaches remain highly computationally intensive. For instance, even basic operations like matrix multiplication require an extensive number of constraints, resulting in significant overhead. In addressing this challenge, we introduce zkVC, which optimizes the ZKP computation for matrix multiplication, enabling rapid proof generation on the server side and efficient verification on the client side. zkVC integrates optimized ZKP modules, such as Constraint-reduced Polynomial Circuit (CRPC) and Prefix-Sum Query (PSQ), collectively yielding a more than 12-fold increase in proof speed over prior methods. The code is available at <https://github.com/UCF-Lou-Lab-PET/zkformer>.

Index Terms—Private and Verifiable Computing, Zero-Knowledge Proof, Machine Learning

I. INTRODUCTION

Zero-knowledge proof (ZKP) [1], [2] is a cryptographic primitive that enables a prover to convince a verifier of the correctness of a computation without revealing the prover’s secret input. ZKP ensures that a proof passes verification only if the computation was performed correctly, guaranteeing both verifiability and privacy. By providing strong guarantees on the integrity of computations while preserving the privacy of the server’s input, ZKP has found broad applications in domains where computational integrity is critical, such as blockchain and verifiable machine learning.

In Figure 1, we illustrate one use case of ZKP in verifiable machine learning. ZKP allows the owner of a proprietary machine learning model, *prover*, to prove to the users, *verifier*, that predictions have been accurately computed by the pre-trained model, without compromising the model’s confidentiality. As depicted, the client-side verifier first sends input data X to the

server, where the server’s prover performs the neural network inference $f(X, W)$ to output a prediction. ZKP is then used to generate a proof for this inference, ensuring $f(X, W)$ ’s validity without revealing the model weights, enabling the client to verify the proof.

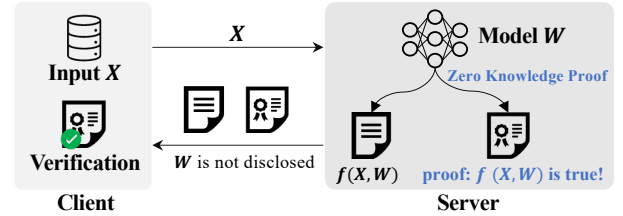


Fig. 1: Example of use case in verifiable and private neural network inference based on Zero-knowledge Proof (ZKP).

While promising, ZKP often leads to large computation overhead in practice, especially when proving matrix multiplication. Proving a single matrix multiplication of dimension $[49, 320] \times [320, 512]$ on a 16-Core CPU with a commonly used ZKP scheme [2] known for efficiency, can take up to ~ 3 minutes. Matrix multiplication is a fundamental operation in numerous applications, including data analysis and machine learning. However, the inefficiencies in current ZKP frameworks make it impractical to directly scale them for real-world scenarios involving massive matrix multiplications. For instance, Transformer-based models [3], [4], widely used in machine learning, rely heavily on extensive matrix multiplications. Proving the correctness of a single inference using a ViT-Base model [4] on the ImageNet dataset [5] results in prohibitively high computational costs, rendering such approaches infeasible.

A series of works have explored how to scale up ZKP-based verifiable computations for real-world applications [6]–[9], where many of them are built upon zk-SNARKs. zk-SNARKs is a branch of general ZKP, which features short proof sizes, fast verification, and non-interactivity and is suitable for cloud computing. The main challenge in reducing the overhead of zk-SNARKs is to reduce the number of constraints. Recent works [6], [9] reduce proving complexity by expressing convolution as polynomial multiplication within a polynomial quadratic arithmetic program (QAP). However, their optimization highly depends on the convolution’s unique feature, which cannot directly extend to general matrix multiplication. The efficient

*Corresponding author: Qian Lou (qian.lou@ucf.edu)

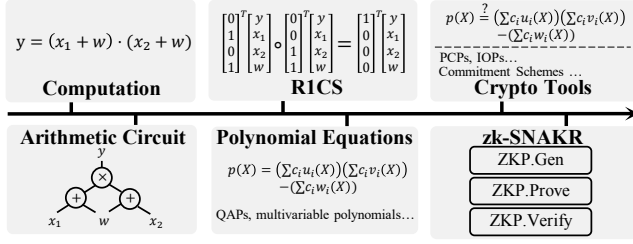


Fig. 2: The workflow of zero-knowledge proof systems.

construction of zk-SNARKs for matrix multiplication remains an open problem.

Proving the correctness of matrix multiplication begins by transforming it into an arithmetic circuit, which is then represented as a QAP. The efficiency of zk-SNARKs relies heavily on the complexity of the QAP, determined by two key factors: (1) the number of constraints, dictated by the multiplication gates in the arithmetic circuit, which also defines the degree of the constraint system, and (2) the number of variables, corresponding to the length of a full assignment to the circuit. Previous work [6] proposed reducing the number of constraints in convolution operations by introducing additional variables, referred to as dummy terms. However, directly applying this approach to matrix multiplication significantly increases the variable count, reducing overall efficiency. Notably, similar to the construction in vCNN [6], our verification process remains succinct and is largely independent of the complexity of the original computation.

Our Contributions. This paper introduces zkVC, an efficient zk-SNARK construction tailored for general operations such as matrix multiplication and their applications in machine learning, including attention-based Transformers. We propose **Constraint-Reduced Polynomial Circuits (CRPC)** to minimize the constraints required for matrix multiplication in ZKP. By transforming matrix multiplication into polynomial multiplication represented in a quadratic arithmetic program (QAP), CRPC reduces the number of constraints from $O(n^3)$ to $O(n)$. Additionally, we introduce **Prefix-Sum Query (PSQ)**, a technique that reduces the number of variables by optimizing the circuit for product accumulation in matrix multiplication. Experimental results demonstrate that the combination of CRPC and PSQ achieves a 12× improvement in proving time for matrix multiplication compared to prior methods. Furthermore, we apply zkVC to verifiable Transformer inference, achieving over a 15× runtime reduction on ViT models compared to baselines without our optimizations.

II. BACKGROUND AND MOTIVATION

ZKP Systems. ZKP is a cryptographic protocol enabling a prover $\mathcal{P}$ to assure a verifier $\mathcal{V}$ of a statement's truth without disclosing anything beyond its validity. In verifiable neural networks, it lets a model owner $\mathcal{P}$ confirm the accuracy of a neural network inference to a client $\mathcal{V}$. Figure 2 outlines a ZKP system's process. To prove a computation like $y = (x_1 + w) \cdot (x_2 + w)$, it is first translated into an arithmetic circuit using addition and multiplication gates. This circuit is then converted

TABLE I: The comparison between zkVC and prior verifiable DNN methods including SafetyNets [14], Keuffner's [15], vCNN [6], VeriML [16], ZEN [17], zkCNN [7], zkML [8] and pvCNN [9].

Schemes	zk.	Non-Inter.	Const. Proof	No Trusted Setup	Trans-formers	Efficient MatMult	zk-ML Codesign
SafetyNets	X	X	X	✓	X	X	X
zkCNN	✓	X	X	✓	X	X	X
Keuffner's	✓	✓	✓	X	X	X	X
vCNN	✓	✓	✓	X	X	X	X
VeriML	✓	✓	✓	X	X	X	X
ZEN	✓	✓	✓	X	X	X	X
zkML	✓	✓	X	X	X	X	X
pvCNN	✓	✓	✓	X	X	X	X
zkVC	✓	✓	✓	✓	✓	✓	✓

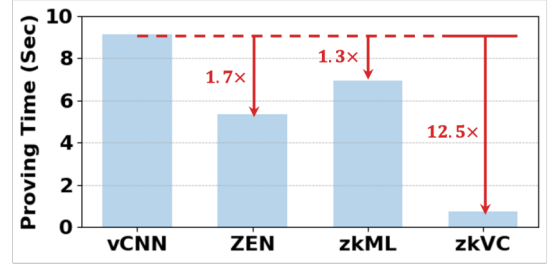


Fig. 3: Proving Time Comparison for Matrix Multiplication with Prior Work.

into a constraint system, such as the Rank-1 Constraint System (R1CS) [1], which generalizes arithmetic circuit satisfiability. In R1CS, additions are represented by row vectors, for instance, $[0, 1, 0, 1]$ for $(x_1 + w)$, while multiplications are encoded using element-wise products. Therefore, proving the original computation's correctness equals satisfying the R1CS. Efficient R1CS checking involves encoding it into polynomials. Schemes like [1], [2] use Quadratic Arithmetic Programs (QAP) to encode R1CS, derived through polynomial interpolation on R1CS instances. Other approaches employ univariate polynomials [10], [11] or multivariate polynomials [12], [13] for encoding.

The verification of R1CS or its polynomial equations requires specific cryptographic tools like groth16 [2] or Spartan [13]. groth16, a widely-used ZKP construction, employs probabilistically checkable proofs (PCPs) and elliptic curve pairings for QAP verification. Spartan, on the other hand, uses Interactive Oracle Proof (IOP) [18] and polynomial commitment schemes [19] for multivariate polynomial checks, notably without requiring a trusted setup. A ZKP construction consists of three Probabilistic Polynomial Time (PPT) algorithms: Gen, Prove, and Verify. Gen creates public parameters from security parameters. Prove uses these parameters, public and private inputs, to produce a proof. Verify, using the public parameters, input, and proof, decides its validity. ZKP ensures proof completeness and soundness, meaning correct computations pass verification, and it is computationally hard for a dishonest prover to validate incorrect computations.

Homomorphic Encryption for Private Computation. Homomorphic encryption (HE) enables computations directly on encrypted data, removing the need for decryption and thus supporting privacy-preserving outsourcing of computa-

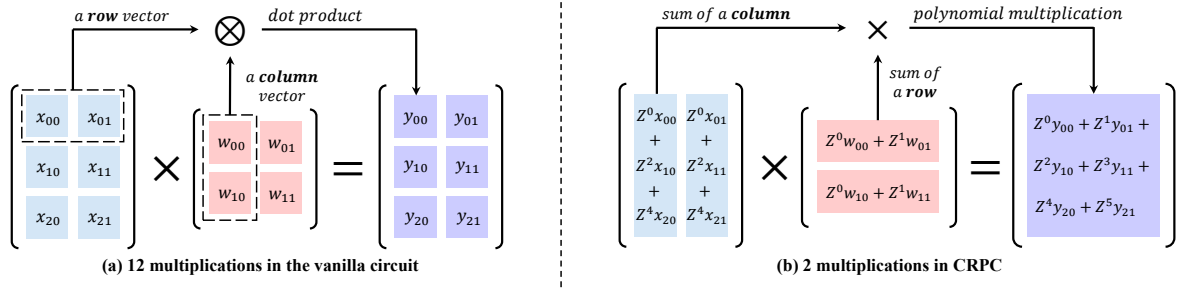


Fig. 4: Comparison of (a) basic constraint circuits with 12 multiplications and our (b) CRPC with 2 multiplications by transforming original matrix multiplication into polynomial multiplications of an intermediate variable Z .

tions [20]–[23]. However, HE alone does not inherently offer computational verifiability as ZKP do [24], [25]. Conversely, while ZKP ensures computational integrity, it lacks the data privacy guarantees provided by HE [26]–[29].

Comparison with Related Work. Table I shows the comparison of our zkVC and related works. SafetyNets [14] lacks zero-knowledge properties, leaving model weight privacy unprotected. Only SafetyNets and zkCNN [7] are interactive, necessitating ongoing communication between prover and verifier. Interactive ZKPs offer quicker proving times but require ongoing exchanges between the prover and verifier. While zero-knowledge polynomial commitment [30] and the Fiat-Shamir heuristic [31] could theoretically make these non-interactive, their security and efficiency impacts are unclear. The need for constant connectivity in interactive setups is a limitation, particularly for clients with limited hardware and power resources [17]. Table I’s third column indicates that in SafetyNets, zkCNN, and zkML’s [8], proof size grows logarithmically with model size, increasing verifier workload. Other schemes maintain a constant proof size. Non-interactive schemes generally require a trusted setup for public parameter generation, but interactive ones like SafetyNets and zkCNN do not. zkVC, using transparent zk-SNARKs such as Spartan, also bypasses the need for a trusted setup. Previous research concentrated solely on CNNs, while zkVC focuses on general matrix multiplication, introducing efficient modules for proving matrix multiplication.

Motivation. Matrix multiplication is essential in many applications but challenging to prove using ZKP. Figure 3 shows that in prior vCNN [6], proving a small matrix multiplication of dimension $[49, 64] \times [64, 128]$ takes as long as 9 seconds. Despite ZEN [17] introducing advanced quantization and zkML’s [8] using a more efficient ZKP method, their improvements in matrix multiplication speed are still limited. This motivates us to design efficient ZKP modules for matrix multiplication. The proposed zkVC achieves a 12.5 $\times$ reduction in proof time compared to the previous vCNN.

III. zkVC DESIGN

A. Constraint-reduced Polynomial Circuits (CRPC)

Matrix multiplication plays a foundational role in various computational tasks, but its efficient representation in QAP remains challenging. As in Figure 4 (a), in vanilla QAP, every

individual multiplication requires a distinct constraint. As every y_{ij} is a dot product between a row vector of X and a column vector of W , we have $y_{00} = x_{00} \cdot w_{00} + x_{01} \cdot w_{10}$, ... and $y_{21} = x_{20} \cdot w_{01} + x_{21} \cdot w_{11}$, where 12 multiplications result in 12 constraints in the QAP. Consider the matrix multiplication $Y = X \times W$ where $X \in \mathbb{R}^{a \times n}$, $W \in \mathbb{R}^{n \times b}$ and $Y \in \mathbb{R}^{a \times b}$. Our insight is that the products in $\{x_{ik} \cdot w_{kj}\}_{k=0}^{n-1}$ for y_{ij} can be encoded in polynomial multiplications. One intuitive transformation is:

$$\begin{aligned} & (y_{00} + y_{01} + y_{10} + y_{11} + y_{20} + y_{21}) \\ &= (x_{00} + x_{10} + x_{20}) \cdot (w_{00} + w_{01}) \\ &+ (x_{01} + x_{11} + x_{21}) \cdot (w_{10} + w_{11}) \end{aligned}$$

When matrix multiplication is satisfied, the above equation is also satisfied. However, the converse is not guaranteed. It is possible that the sum of all y_{ij} is correct while individual y_{ij} is not. Essentially, this transformation ensures completeness but compromises the soundness required by zk-SNARKs.

As prior work’s approach [6] suggests, it is possible to encode the matrix multiplication in one single polynomial multiplication, if the coefficients are properly arranged. Another possible transformation is:

$$\begin{aligned} & (Z^1 y_{00} + Z^3 y_{01} + Z^5 y_{10} + Z^7 y_{11} + Z^9 y_{20} + Z^{11} y_{21}) \\ & \neq (Z^1 x_{00} + Z^0 x_{01} + Z^5 x_{10} + Z^4 x_{11} + Z^9 x_{20} + Z^8 x_{21}) \\ & \quad \cdot (Z^0 w_{00} + Z^2 w_{01} + Z^1 w_{10} + Z^3 w_{11}) \end{aligned}$$

where $y_{00} = x_{00} \cdot w_{00} + x_{01} \cdot w_{10}$ is incorporated within $Z^1 y_{00} = Z^1 (x_{00} \cdot w_{00} + x_{01} \cdot w_{10})$. However, this transformation is not strictly equivalent. Firstly, the equation itself is not satisfied. The polynomial multiplication results in a lot of superfluous terms such as $Z^0 x_{01} w_{01}$, which does not belong to any y_{ij} . Secondly, to equate the two sides, all these dummy terms need to be included, which leads to increased number of variables and makes the circuit hard to prove.

While the above two transformations can reduce the number of multiplications, they cannot ensure the integrity of matrix multiplication. We propose CRPC to address this issue. Our insight is that the products in $\{x_{ik} \cdot w_{kj}\}_{k=0}^{n-1}$ for y_{ij} result only from the multiplications between elements in the k_{th} column in X and elements in k_{th} row in W . As shown in Figure 4 (b), we transform each column of X and each row of W into polynomials of a random intermediate variable Z , for example,

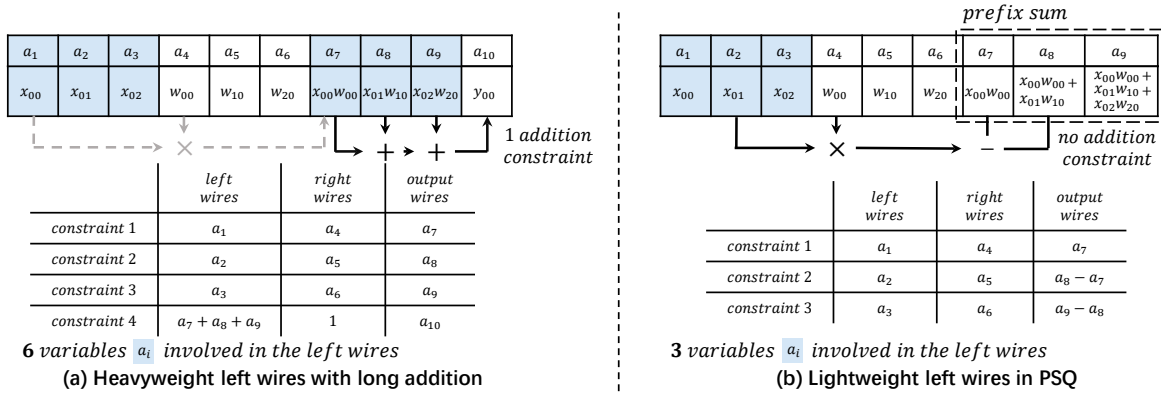


Fig. 5: Comparison: (a) traditional long addition with 6 left-wire variables vs. (b) our PSQ using only 3 variables.

the first column of X is $X_0(Z) = Z^0 x_{00} + Z^2 x_{10} + Z^4 x_{20}$. The matrix Y is converted accordingly. We have:

$$\begin{aligned} & (Z^0 y_{00} + Z^1 y_{01} + Z^2 y_{10} + Z^3 y_{11} + Z^4 y_{20} + Z^5 y_{21}) \\ &= (Z^0 x_{00} + Z^2 x_{10} + Z^4 x_{20}) \cdot (Z^0 w_{00} + Z^1 w_{01}) \\ &+ (Z^0 x_{01} + Z^2 x_{11} + Z^4 x_{21}) \cdot (Z^0 w_{10} + Z^1 w_{11}) \end{aligned}$$

where only 2 multiplications are needed. We generalize CRPC's transformation for matrix multiplication $Y^{a \times b} = X^{a \times n} \times W^{n \times b}$ as:

$$\sum_{j=0}^{b-1} \sum_{i=0}^{a-1} Z^{ib+j} y_{ij} = \sum_{k=0}^{n-1} \left(\sum_{i=0}^{a-1} Z^{ib} x_{ik} \right) \left(\sum_{j=0}^{b-1} Z^j w_{kj} \right)$$

where only n constraints are needed, where as $a \cdot b \cdot n$ constraints are needed in vanilla circuits.

Polynomial multiplication offers significant advantages in computational efficiency, especially when representing matrix multiplication in ZKP protocols. Our proposed CRPC reduces the constraint complexity from $O(n^3)$ to $O(n)$ for matrix multiplication. The proving efficiency of zk-SNARKs systems is thereby improved by a significant margin. The overall proving time of ZKP-based matrix multiplication of different sizes in Transformer layers can be $7 \sim 9\times$ faster.

B. Prefix-Sum Query (PSQ)

The CRPC method notably decreases the constraint count needed for matrix multiplication representation. Additionally, it is observed that the number of left wires influences proving performance. We propose PSQ to reduce the number of left wires. Our insight is that, although an arbitrary number of additions can be encapsulated within a single constraint, a prolonged sequence of additions can result in a considerable computational overhead, leading to a large number of left wires number, as is shown in Figure 5(a). Consider a dot product in matrix multiplication, represented as $y_{00} = x_{00} \cdot w_{00} + x_{01} \cdot w_{10} + x_{02} \cdot w_{20}$. This calculation requires 4 constraints. The initial three constraints calculate the intermediate products: $x_{00}w_{00}$, $x_{01}w_{10}$, and $x_{02}w_{20}$, which are assigned to variables a_7 , a_8 , and a_9 . To achieve the final result y_{00} , one more addition is required, incorporating the three intermediate variables a_7 , a_8 , and a_9 in the left wires. Consequently, this approach uses 6

left variables/wires. Heavy left wires in large matrices can be computationally demanding. Then, we introduce PSQ to avoid extended additions in matrix multiplication, as illustrated in 5 (b). Instead of holding actual intermediate product values in a_7 , a_8 , and a_9 , we record their prefix sums. Specifically, $a_7 = x_{00}w_{00}$, $a_8 = a_7 + x_{01}w_{10}$, $a_9 = a_8 + x_{02}w_{20}$.

Here, the final result y_{00} is directly available in a_9 , eliminating the need for an additional constraint for the long addition. PSQ reduces the left wire variables to only 3, making it a more efficient approach for both verification and proving stages.

The proposed PSQ effectively reduces the complexity. Consider the general matrix multiplication $Y^{a \times b} = X^{a \times n} \times W^{n \times b}$. There are $(a \cdot n + a \cdot b \cdot n)$ variables involved, where the $(a \cdot b \cdot n)$ intermediate products make proving rather complex. With PSQ, the proving is only associated with the $(a \cdot n)$ variables, and the complexity is reduced from $O(n^3)$ to $O(n^2)$. PSQ effectively contributes to a lightweight R1CS, and the computation of the R1CS thus becomes significantly more efficient. Specifically, the cost of computing the R1CS is reduced by approximately 70% during the proving phase. By building on the foundation of CRPC, PSQ further reduces the proving cost for matrix multiplication by 30%. This results in a total speedup of $12\times$ for the proving time.

C. Nonlinear-Function Approximation

CRPC and PSQ significantly improves the efficiency of proving matrix multiplication with ZKP. To demonstrate the efficiency of our zkVC, we apply it to verifiable Transformer inference. To verify the correctness of computation in transformers, we need to verify the non-linear functions like *SoftMax*, and GeLU with ZKP. However, ZKP can not directly support these non-arithmetic functions. We design accurate arithmetic approximations for these complex non-arithmetic functions. Given a vector $x \in \mathbb{R}^d$, the *SoftMax* function is defined as $SoftMax_i(x) = e^{x_i} / \sum_{j \in [d]} e^{x_j}$. The main challenge is to accurately express the exponential function in ZKP constraints. Although the exponential function can not be directly represented by addition and multiplication, it is possible to closely approximate the exponential function on negative inputs. Based on this idea, we show how the SoftMax function is verified in our design as follows.

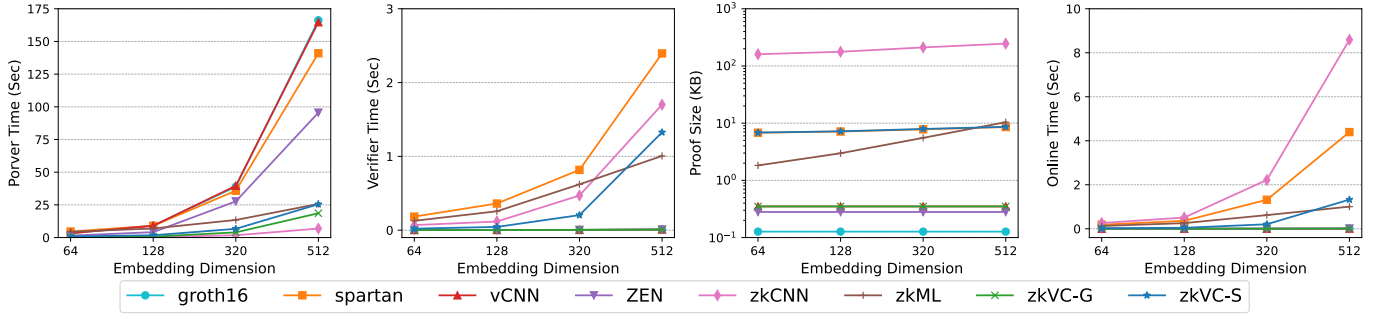


Fig. 6: A comparison of zkVC and prior works on matrix multiplication. zkVC leads in proving time of all non-interactive methods, close to interactive zkCNN, and excels in verification time, proof size, as well as online verification duration.

Computing $x_{\max}$. We first normalize the input vector x by $(x - x_{\max})$, where $x_{\max}$ is the max element in x . The subtraction can easily be encoded in ZKP circuits. To verify the max computation, we check two constraints: (1) $x_{\max} \geq x_j$ for all $j \in [d]$ and (2) $\prod_j (x_{\max} - x_j) = 0$. The first constraint ensures $x_{\max}$ is greater than all other values in x and the second constraint ensures $x_{\max}$ is indeed one of the values from x . Since ZKP supports comparison operations by bit-decomposition [7], these constraints are compatible with ZKP.

Approximating e^x on negative inputs. After normalizing x by $(x - x_{\max})$, all elements in the resulting vector are negative. It is evident that computing $\text{SoftMax}(x - x_{\max})$ is equivalent to computing $\text{SoftMax}(x)$. Thus, the SoftMax function can be computed as $\text{SoftMax}_i(x) = e^{x_i - x_{\max}} / \sum_{j \in [d]} e^{x_j - x_{\max}}$, where only the exponential function on negative inputs is needed to compute. We approximate e^x on negative inputs using the Taylor series:

$$e^x \approx \begin{cases} 0, & \text{if } x < T \\ (1 + x/2^n)^{2^n}, & \text{if } x \in [T, 0]. \end{cases}$$

where T is the pre-defined threshold deciding the clipping branch. We apply a two-bit decomposition to compare x with T and compute the division by 2^n . The power-to- 2^n computation is essentially a series of multiplications that can be easily encoded in ZKP circuits.

Putting together, we verify the SoftMax function in ZKP via a close approximation using three sets of bit decomposition and two sets of multiplication. The GELU activation function is used in NLP transformers such as BERT. The GELU function is defined as $\text{GELU}(x) = 0.5x \left(1 + \tanh\left[\sqrt{2/\pi}(x + 0.044715x^3)\right] \right)$. Similar to the exponential function in SoftMax , the Tanh function can not be directly represented by addition and multiplication gates. We use polynomial approximation to represent the GELU function efficiently. Specifically, we use $\text{GELU}(x) \approx x^2/8 + x/4 + 1/2$.

IV. EXPERIMENTAL METHODOLOGY

Models and Datasets. In our experiments, we explored both computer vision and language transformers. For computer vision, we tested three vision transformer architectures on different datasets, adapting settings from [32], [33] for small datasets. On CIFAR-10, the ViT configuration was 7 layers, 4

heads, a hidden dimension of 256, and a patch size of 4. For Tiny-ImageNet, we used 9 layers, 12 heads, a hidden dimension of 192, and a patch size of 4. On ImageNet, a hierarchical architecture from [34], [35] was implemented with 12 layers and 4 stages, with embedding dimensions of 64, 128, 320, 512. For NLP, we chose a BERT model with 4 layers, 4 heads, and an embedding dimension of 256. This model was fine-tuned and assessed on GLUE benchmarks [36], including MNLI, QNLI, SST-2, and MRPC tasks.

Implementation Details. In our experiments, we utilized groth16 from libsnark [37] and Spartan [13] for the ZKP backend, referred to as zkVC-G and zkVC-S, respectively. These cryptographic tests were conducted on AMD Ryzen Threadripper PRO 3955WX 16-Core CPU systems with 128GB RAM, running Ubuntu 22.04.1. Transformer model experiments were performed on NVIDIA GeForce RTX 3090 GPUs. For Transformer architectures, zkVC was built on ViT [4] and MetaFormer's frameworks [35], incorporating efficient token mixers like scaling attention modules [38], [39], linear transformation modules [40], average pooling, and differentiable NAS [41]. We employed the quantization technique from [42] for converting model parameters to integer formats. This marks the first instance of verifiable Transformers being tested on the ImageNet dataset [5].

V. RESULTS

A. Micro-benchmarks

Matrix Multiplication Benchmark. Figure 6 shows a comparison of zkVC with previous works on matrix multiplication benchmarks. The dimensions of the matrices are set according to the embedding layers in ViT [4], i.e., $[\text{\#tokens}, \text{dim}_1] \times [\text{dim}_1, \text{dim}_2]$. The number of tokens are set to 49 for simplicity. For example, when the embedding dimension is 128, the dimension of matrix multiplication is $[49, 64] \times [64, 128]$. For non-interactive schemes, zkVC-G is based on groth16 [2], and zkVC-S on Spartan [13]. vCNN [6] and ZEN [17] also use groth16, while Kang's [8] employs halo2 [43]. The interactive scheme included is zkCNN [7], with vanilla groth16 and Spartan serving as baselines. zkVC significantly enhances proving time, achieving 5 to 12 times faster performance than the groth16 and Spartan baselines. Among non-interactive options, zkVC-G stands out for its proving efficiency. Though

TABLE II: Ablation study on matrix multiplication microbenchmark.

CRPC	PSQ	groth16		Spartan	
		Prove(s)	Verify(s)	Prove(s)	Verify(s)
X	X	9.12	0.002	9.04	0.36
X	✓	8.69	0.002	8.95	0.32
✓	X	1.01	0.002	1.79	0.08
✓	✓	0.73	0.002	1.75	0.05

TABLE III: Comparison of various token mixers with our zkVC on ViT Models. SoftApprox. includes approximated SoftMax, Softfree-S (scaling), Softfree-P (pooling), with $\mathcal{P}_G$ for Groth16 and $\mathcal{P}_S$ for Spartan.

Dataset	Model	Top1(%)	$\mathcal{P}_G$ (s)	$\mathcal{P}_S$ (s)
Cifar-10	SoftApprox.	93.5	725.2	1006.2
	SoftFree-S	88.3	568.4	742.8
	SoftFree-P	75.1	262.7	300.6
	zkVC	91.6	458.6	591
Tiny ImageNet	SoftApprox.	60.5	1609.6	2197.4
	SoftFree-S	51.4	1004.9	1348.8
	SoftFree-P	42.7	443.7	503.6
	zkVC	55.8	879.3	1161.4
ImageNet	SoftApprox.	81	10700	12857.7
	SoftFree-S	78.5	4521.3	5812.7
	SoftFree-P	77.2	2904	3667.8
	zkVC	80.3	3457.1	4417.1

zkCNN is about twice as fast in proving compared to zkVC-G, it requires interaction and suffers from slower verification and larger proof sizes. Specifically, zkCNN’s verification is up to 200 times slower and its proofs are 1 to 2 orders of magnitude larger than those of zkVC. The online time means the time that the client and server need to maintain online during the proving. zkCNN also demands additional verifier online time due to its interactive nature.

Technical Ablation Study. In Table II, we present the latency results from a matrix multiplication microbenchmark to highlight the effectiveness of the proposed CRPC and PSQ in transformer patch embedding layers. CRPC significantly reduces proving time, boosting the groth16 backend by approximately 9 $\times$ and the Spartan backend by around 5 $\times$. While groth16 maintains a consistent verification time, CRPC cuts Spartan’s verification time by about 4 $\times$. The application of PSQ further speeds up groth16, achieving up to a 12 $\times$ faster rate. Directly applying PSQ to Groth16 can also improve proving time, but the enhancement is more significant when PSQ is coupled with CRPC. However, PSQ’s impact on Spartan’s proving time is minimal, this is because CRPC already reduces the number of constraints for both backends, while PSQ primarily simplifies specific complex queries for groth16.

B. End-to-end Performance

Vision. Table III illustrates zkVC’s accuracy-latency balance on three popular vision datasets. For smaller datasets like Cifar-10 and Tiny ImageNet, using average pooling instead of SoftMax self-attention notably reduces accuracy. Although scaling attention models are more accurate, their efficiency gain in ZKP proving is minimal due to low-resolution images (e.g., 32×32 in Cifar-10) and fewer transformer input tokens (e.g.,

TABLE IV: Comparison of various token mixers with our zkVC on NLP Models. SoftFree-L denotes a model using linear transformation for token mixing.

Model	Acc. on Tasks(%)				$\mathcal{P}_G$ (s)	$\mathcal{P}_S$ (s)
	MNLI	QNLI	SST-2	MRPC		
SoftApprox.	74.5	83.9	85.8	71.2	1299.5	1793.3
SoftFree-S	72.7	81.1	85.2	70.4	917.1	1201.4
SoftFree-L	67.3	75.3	84.5	68.7	680.8	782.0
zkVC	70.8	80.2	84.7	69.3	798.9	992.2

64 tokens for 4-size patches). Average pooling alone struggles with low-res images, and scaling attention alone shows limited efficiency for short token sequences. zkVC, blending *SoftMax* self-attention with *SoftMax*-free options, achieves around 40% faster proving on Cifar-10 with under 2% accuracy loss, and about 50% faster on Tiny ImageNet with less than 5% accuracy loss.

High-resolution ImageNet images, with many input tokens (e.g., 3136 for a 224×224 image with patch size 4), strain SoftMax self-attention’s quadratic complexity. Scaling attention’s linear complexity better manages these long sequences. SoftMax-free methods, like scaling attention and average pooling, excel on larger datasets, speeding up computations by 60% to 70% with less than 5% accuracy loss. zkVC achieves similar speedups with under 1% accuracy loss, likely due to reintegrating SoftMax self-attention in later transformer layers with shorter token sequences.

NLP. Table IV shows zkVC’s evaluation on NLP transformers like BERT. Linear transformation improves efficiency by 50% but can drop accuracy by up to 7% on MNLI. Scaling attention increases proving efficiency by 30%, with better, more stable performance across tasks. zkVC is about 15% faster than scaling attention models while being around 3% more accurate on average compared to linear transformation. The results indicate that replacing all *SoftMax* attention with SoftMax-free alternatives isn’t always ideal for accuracy and latency demands. zkVC, using our planner, combines a hybrid transformer architecture. It matches *SoftMax*-centric models in accuracy and outperforms Linear Attention models in latency.

VI. CONCLUSION

In this paper, we introduced *zkVC*, an efficient zk-SNARK construction designed to optimize matrix multiplication verification. Traditional ZKP approaches for verifying matrix multiplication often require an excessive number of constraints, leading to high computational overhead. *zkVC* addresses this challenge by leveraging CPRC to minimize constraints and PSQ to reduce variables, achieving a 12 $\times$ improvement in proof efficiency over prior methods. We further demonstrated *zkVC*’s effectiveness in verifiable Transformer inference, verifying the integrity of ViT models efficiently. Given that matrix multiplication underpins a wide range of applications, we believe *zkVC* offers a significant step forward in enhancing the computational integrity of these applications.

REFERENCES

- [1] B. Parno, J. Howell, C. Gentry, and M. Raykova, “Pinocchio: Nearly practical verifiable computation,” *Communications of the ACM*, vol. 59, no. 2, pp. 103–112, 2016.
- [2] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II* 35. Springer, 2016, pp. 305–326.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [6] S. Lee, H. Ko, J. Kim, and H. Oh, “vcnn: Verifiable convolutional neural network based on zk-snarks,” *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [7] T. Liu, X. Xie, and Y. Zhang, “Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985.
- [8] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun, “Scaling up trustless dnn inference with zero-knowledge proofs,” 2022.
- [9] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J.-N. Liu, “pvcnn: Privacy-preserving and verifiable convolutional neural network testing,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2218–2233, 2023.
- [10] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, “Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2111–2128.
- [11] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, “Marlin: Preprocessing zk-snarks with universal and updatable srs,” in *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 2020, pp. 738–768.
- [12] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish, “Doubly-efficient zk-snarks without trusted setup,” in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 926–943.
- [13] S. Setty, “Spartan: Efficient and general-purpose zk-snarks without trusted setup,” in *Annual International Cryptology Conference*. Springer, 2020, pp. 704–737.
- [14] Z. Ghodsi, T. Gu, and S. Garg, “Safetytnets: Verifiable execution of deep neural networks on an untrusted cloud,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [15] J. Keuffer, R. Molva, and H. Chabanne, “Efficient proof composition for verifiable computation,” in *Computer Security - 23rd European Symposium on Research in Computer Security, ESORICS 2018, Barcelona, Spain, September 3–7, 2018, Proceedings, Part I*, ser. Lecture Notes in Computer Science, J. López, J. Zhou, and M. Soriano, Eds., vol. 11098. Springer, 2018, pp. 152–171.
- [16] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, and B. Feng, “Veriml: Enabling integrity assurances and fair payments for machine learning as a service,” *IEEE Trans. Parallel Distributed Syst.*, vol. 32, no. 10, pp. 2524–2540, 2021.
- [17] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu, “Zen: An optimizing compiler for verifiable, zero-knowledge neural network inferences,” *Cryptology ePrint Archive*, Paper 2021/087, 2021.
- [18] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *Theory of Cryptography: 14th International Conference, TCC 2016-B, Beijing, China, October 31–November 3, 2016, Proceedings, Part II* 14. Springer, 2016, pp. 31–60.
- [19] B. Bünz, B. Fisch, and A. Szeponiec, “Transparent snarks from dark compilers,” in *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 2020, pp. 677–706.
- [20] Q. Lou and L. Jiang, “She: A fast and accurate deep neural network for encrypted data,” *Advances in neural information processing systems*, vol. 32, 2019.
- [21] M. Zheng, Q. Lou, and L. Jiang, “Primer: Fast private transformer inference on encrypted data,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [22] Q. Lou, Y. Shen, H. Jin, and L. Jiang, “Safenet: A secure, accurate and fast neural network inference,” in *International Conference on Learning Representations*, 2021.
- [23] Q. Lou and L. Jiang, “Hemet: A homomorphic-encryption-friendly privacy-preserving mobile neural network architecture,” in *International conference on machine learning*. PMLR, 2021, pp. 7102–7110.
- [24] Q. Lou, M. Santriagi, A. W. B. Yudha, J. Xue, and Y. Solihin, “vfhe: Verifiable fully homomorphic encryption with blind hash,” *arXiv preprint arXiv:2303.08886*, 2023.
- [25] M. H. Santriagi, J. Xue, Y. Zhang, Q. Lou, and Y. Solihin, “Dataseal: Ensuring the verifiability of private computation on encrypted data,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2024, pp. 78–78.
- [26] M. Kumar, J. Xue, M. Zheng, and Q. Lou, “Tfhe-coder: Evaluating llm-agentic fully homomorphic encryption code generation,” *arXiv preprint arXiv:2503.12217*, 2025.
- [27] Y. Zhang, M. Zheng, Y. Shang, X. Chen, and Q. Lou, “Heprune: Fast private training of deep neural networks with encrypted data pruning,” in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 51 063–51 084.
- [28] Y. Zhang, J. Xue, M. Zheng, M. Xie, M. Zhang, L. Jiang, and Q. Lou, “Cipherprune: Efficient and scalable private transformer inference,” *arXiv preprint arXiv:2502.16782*, 2025.
- [29] X. Deng, S. Fan, Z. Hu, Z. Tian, Z. Yang, J. Yu, D. Cao, D. Meng, R. Hou, M. Li *et al.*, “Trinity: A general purpose fhe accelerator,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 338–351.
- [30] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish, “Doubly-efficient zk-snarks without trusted setup,” in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 926–943.
- [31] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1986, pp. 186–194.
- [32] A. Hassani, S. Walton, N. Shah, A. Abuduweili, J. Li, and H. Shi, “Escaping the big data paradigm with compact transformers,” *arXiv preprint arXiv:2104.05704*, 2021.
- [33] W. Zeng, M. Li, W. Xiong, W. Lu, J. Tan, R. Wang, and R. Huang, “Mpcvit: Searching for mpc-friendly vision transformer with heterogeneous attention,” *arXiv preprint arXiv:2211.13955*, 2022.
- [34] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.
- [35] W. Yu, M. Luo, P. Zhou, C. Si, Y. Zhou, X. Wang, J. Feng, and S. Yan, “Metaformer is actually what you need for vision,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 819–10 829.
- [36] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” *arXiv preprint arXiv:1804.07461*, 2018.
- [37] “libsark,” <https://github.com/scipr-lab/libsark>.
- [38] Z. Shen, M. Zhang, H. Zhao, S. Yi, and H. Li, “Efficient attention: Attention with linear complexities,” 2020.
- [39] X. Wang, R. Girshick, A. Gupta, and K. He, “Non-local neural networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7794–7803.
- [40] J. Lee-Thorp, J. Ainslie, I. Eckstein, and S. Ontanon, “Fnet: Mixing tokens with fourier transforms,” *arXiv preprint arXiv:2105.03824*, 2021.
- [41] H. Liu, K. Simonyan, and Y. Yang, “Darts: Differentiable architecture search,” *arXiv preprint arXiv:1806.09055*, 2018.
- [42] M. Wang, S. Rasoulizadeh, P. H. W. Leong, and H. K.-H. So, “NITI: Training integer neural networks using integer-only arithmetic,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 3249–3261, nov 2022.
- [43] S. Bowe, J. Grigg, and D. Hopwood, “Recursive proof composition without a trusted setup,” *Cryptology ePrint Archive*, 2019.